Solution: First Missing Positive

Problem Statement

Given an unsorted array `nums` containing positive and negative integers, return the `smallest missing` positive integer value.

Note: You must implement algorithm in $o(n)$ time and uses $o(1)$ space.

Examples

- **Example 1**:
    - **Input**: [5, 3, -1, 8, 4, 2]
    - **Expected Output**: **1**
    - **Justification**: The first missing positive integer in the array is 1.
- **Example 2**:
    - **Input**: [2, 5, -7, 3, 9, 4, 6, 1]
    - **Expected Output**: **7**
    - **Justification**: The array contains all integers from 1 to 6, so the first missing positive integer is 7.
- **Example 3**:
    - **Input**: [-4, -3, -2, -1]
    - **Expected Output**: **1**
    - **Justification**: Since there are no positive integers, the first missing positive integer is 1.

Solution

To solve this problem, we'll employ an in-place algorithm to rearrange the elements such that each positive integer (i) is placed at the index (i-1), ignoring negative numbers and numbers larger than the array's length. This approach ensures that if a positive integer is missing, its corresponding index will not hold the correct value.

Our solution is effective because it allows us to identify the missing positive integer with a single pass through the array to rearrange the elements, followed by another pass to find the first index not holding its corresponding positive integer. This method avoids the need for extra space for another data structure, making it efficient in terms of both time and space complexity.

Step-by-step Algorithm

- **Initialize** a variable `n` to store the length of the array.
- **First Pass**: Iterate over the array.
    - For each element `num` at index `i`, while `num` is within the range `[1, n]` and the element at `num-1` is not `num`, swap `num` with the element at `num-1`.
- **Second Pass**: Iterate over the array again.
    - If the element at index `i` is not `i+1`, return `i+1`.
- If all positions contain the correct integers, return `n+1` to indicate that the first missing positive is just after the last index.

Algorithm Walkthrough

Let's consider the input: `[2, 5, -7, 3, 9, 4, 6, 1]`

Step 1: **Start with the first element** (**2**)

- **Element is 2**; it should be at index 1 (since we are aiming for (i) at (i-1) index, considering 1-based indexing for the target positions).
- Swap 2 with the element at its target index (1), which is 5.
- **Array after swap**: `[5, 2, -7, 3, 9, 4, 6, 1]`

Step 2: **Look at the element now at index 0** (**5**)

- **Element is 5**; it should be at index 4.
- Swap 5 with the element at index 4 (9).
- **Array after swap**: `[9, 2, -7, 3, 5, 4, 6, 1]`

Step 3: **Process the element now at index 0** (**9**)

- **Element is 9**; it's out of range for our placement (since there is no index 8), so we leave it as is.

Step 4: **Move to the second element** (**2**)

- **Element is 2**; it should be at index 1.
- Swap 2 with the element at index 1 (itself in this case, no action needed).
- **Array state remains**: `[9, 2, -7, 3, 5, 4, 6, 1]`

Step 5: **Move to the third element** (**-7**)

- **Element is -7**; negative numbers are ignored.

Step 6: **Move to the fourth element** (**3**)

- **Element is 3**; it should be at index 2.
- Swap 3 with the element at index 2 (-7).
- **Array after swap**: `[9, 2, 3, -7, 5, 4, 6, 1]`

Step 7: **Move to the element now at index 3** (**-7**)

- **Element is -7**; negative numbers are ignored.

Step 8: **Move to the fifth element** (**5**)

- **Element is 5**; it should be at index 4.
- It's already in the correct position.

Step 9: **Move to the sixth element** (**4**)

- **Element is 4**; it should be at index 3.
- Swap 4 with the element at index 3 (-7).
- **Array after swap**: `[9, 2, 3, 4, 5, -7, 6, 1]`

Step 10: **Move to the seventh element** (**6**)

- **Element is 6**; it should be at index 5.
- Swap 6 with the element at index 5 (-7).
- **Array after swap**: `[9, 2, 3, 4, 5, 6, -7, 1]`

Step 11: **Move to the third element** (**-7**)

- **Element is -7**; negative numbers are ignored.

Step 12: **Move to the eighth element** (**1**)

- **Element is 1**; it should be at index 0.
- Swap 1 with the element at index 0 (9).
- **Array after swap**: `[1, 2, 3, 4, 5, 6, -7, 9]`

Final Array State

```
[1, 2, 3, 4, 5, 6, -7, 9]
```

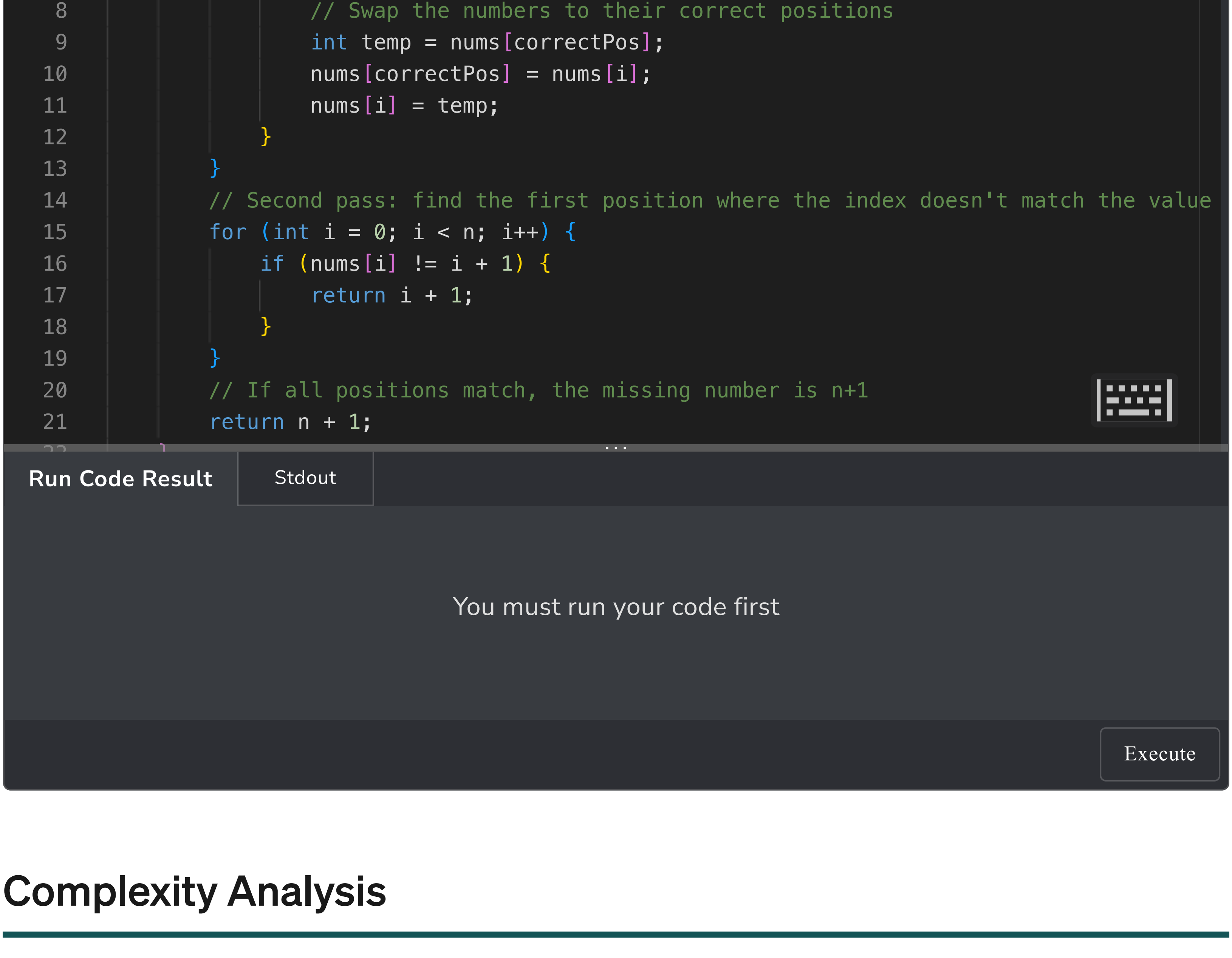
Finding the First Missing Positive

- Start from the beginning of the array, checking if the element matches its index + 1.
- All elements from 1 to 6 are correctly placed.
- The first missing positive integer, given our repositioning, is at index 6 (considering 0-based indexing), where we expect to find 7, but we find -7.

Result

After rearranging the array and checking for the first non-matching index, the first missing positive integer for the array `[2, 5, -7, 3, 9, 4, 6, 1]` is determined to be **7**.

Code



```java
public class Solution {
    public int firstMissingPositive(int[] nums) {
        int n = nums.length;
        // First pass: place each number in its right place, if possible
        for (int i = 0; i < n; i++) {
            while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i]) {
                int correctPos = nums[i] - 1;
                // Swap the numbers to their correct positions
                int temp = nums[correctPos];
                nums[correctPos] = nums[i];
                nums[i] = temp;
            }
        }
        // Second pass: find the first position where the index doesn't match the value
        for (int i = 0; i < n; i++) {
            if (nums[i] != i + 1) {
                return i + 1;
            }
        }
        // If all positions match, the missing number is n+1
        return n + 1;
```

Complexity Analysis

Time Complexity

- $O(n)$: The algorithm passes through the array a constant number of times (specifically, two passes). Each pass is linear in terms of the number of elements in the array. Therefore, the total operations are proportional to the number of elements, (n).

Space Complexity

- $O(1)$: The space complexity is constant because the algorithm rearranges the elements in place without using any additional data structures that grow with the size of the input array.

← Previous
First Missing Positive (Hard)

Next →
Largest Rectangle In Histogram

☐ Mark as Completed